SMASHKARTS OUTDOOR EVENT

Lenovo Essential FHD Webcam FHDWC300 • 1920×1080 @ 30 FPS MJPEG • 18 × 10 ft outdoor arena

Static ArUco corner markers 0–3 • HSV colour blob cars • 3D roof-parallax correction • Windows 11 / VS Code

Engineering test manual | Updated workflow and prototype scripts

START HERE — Scenario Selection

Situation	Execute
First setup / new or larger outdoor track	Sequence A: full rebuild (Section 11)
Only camera position changed; same track and corner tags	Sequence B: recalibrate only, then verify tracking
Same setup next session/day	Sequence C: quick camera check and HSV sunlight check
Outdoor lighting / time-of-day changed	Run camera/pick_hsv_color.py and update CAR_CONFIGS; verify tracking
Car roof colour or number of cars changed	Update CAR_CONFIGS in both tracking scripts; retest overlap and contour area

0. Hardware, arena & environment

Camera: Lenovo Essential FHD Webcam, model FHDWC300, one overhead camera mounted at 12.0 ft (3658 mm), pointing vertically down. Required capture mode: 1920×1080 at 30 FPS using MJPEG and DirectShow. A requested mode is not guaranteed: verify actual frame dimensions/FPS on the camera used.

Optics supplied for planning: 95° diagonal FOV; 86.26° horizontal × 56.60° vertical on 16:9 sensor. At 12 ft, estimated raw footprint 22.48 × 12.92 ft (6852 × 3938 mm), about 85.4 px/ft or 2.80 px/cm at 1920×1080. Verify footprint physically; lens/FOV and driver crop can differ.

Arena: outer crash barriers 18 × 10 ft (5486 × 3048 mm); conservative alternative 16 × 9 ft. Matte charcoal/black floor (#2B2B2B), matte white 2 in / 50 mm boundary lines inset at least 6 in / 150 mm from crash walls. No glossy tape over panel seams. Direct sunlight creates changing shadows and glare; test at event time.

Static corner tags: DICT_4X4_50; ID 0 TL, 1 TR, 2 BR, 3 BL. Each black marker is 18 × 18 cm on 23 × 23 cm rigid matte off-white/light-grey cardstock (#D8D8D8). Place flat on ground outside barriers at four corners along left/right long flanks. Ensure all four are visible and not clipped.

Cars: about 30 × 16 × 12 cm. NO moving ArUco tags. Car 1 deep matte crimson/red, Car 2 deep matte cobalt/royal blue; optional Car 3 deep matte forest green. Avoid fluorescent/neon/yellow colours. HSV varies in sunlight; measure with the picker and validate in full track view.

```
gazebo/
  .venv/
  camera/
    camera_detect.py
    camera_test.py
    pick_hsv_color.py
  track/
    capture_track.py
    create_mask.py
    extract_pixels.py
    draw_centerline.py
    calculate_progress.py
    test_progress.py
    recalibrate_camera.py
    live_tracking.py
    lap_counter.py
    track_reference.png
    perspective_matrix.npy
    track_mask.png
    track_pixels.npy
    centerline_points.npy
    dense_centerline.npy
    cumulative_distance.npy
    lap_length.npy
```

Open gazebo in VS Code. Run all PowerShell commands from the gazebo project root. Close each camera window with Q before launching another script. Camera index may differ by laptop/USB connection.

```
.\.venv\Scripts\python.exe -m pip install --upgrade numpy opencv-contrib-python
.\.venv\Scripts\python.exe -c "import cv2; print(cv2.__version__, hasattr(cv2, 'aruco'))"
```

Expected final value True for aruco. Do not install opencv-python and opencv-contrib-python together in the same environment. If switching packages, uninstall both first, then install opencv-contrib-python.

1. Camera discovery, 1080p verification & outdoor HSV picker

1.1 File: camera/camera_detect.py

Lists indices that return a frame. Cover the external webcam lens to identify its index.

```
.\.venv\Scripts\python.exe camera\camera_detect.py
```

```
import cv2
for i in range(8):
    cap = cv2.VideoCapture(i, cv2.CAP_DSHOW)
    if cap.isOpened():
        ok, frame = cap.read()
        if ok: print(f"Index {i}: {frame.shape[1]}x{frame.shape[0]}")
    cap.release()
```

1.2 File: camera/camera_test.py

Forces the requested mode and prints actual frame dimensions. Driver properties may be unsupported or may have different scales; trust the measured output, not the requested values.

```
.\.venv\Scripts\python.exe camera\camera_test.py
```

```
import cv2
CAMERA_INDEX = 0  # set from camera_detect.py
def open_camera(index):
    cap = cv2.VideoCapture(index, cv2.CAP_DSHOW)
    if not cap.isOpened(): raise RuntimeError("Camera could not open")
    cap.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc(*'MJPG'))
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1920)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 1080)
    cap.set(cv2.CAP_PROP_FPS, 30)
    cap.set(cv2.CAP_PROP_AUTO_WB, 0)
    cap.set(cv2.CAP_PROP_WB_TEMPERATURE, 5500)
    cap.set(cv2.CAP_PROP_BRIGHTNESS, 100)
    return cap
cap = open_camera(CAMERA_INDEX)
print("Requested 1920x1080@30 MJPG; checking actual frames")
while True:
    ok, frame = cap.read()
    if not ok: print("Frame read failed"); break
    print(f"Actual frame: {frame.shape[1]}x{frame.shape[0]}", end="\r")
    cv2.imshow("FHDWC300 1080p camera", cv2.resize(frame,(960,540)))
    if cv2.waitKey(1) & 0xFF == ord('q'): break
cap.release(); cv2.destroyAllWindows()
```

1.3 File: camera/pick_hsv_color.py

Click a roof pixel in the live feed. It prints sampled HSV and a conservative suggested band. Repeat in sun and shadow; do not blindly use one pixel range as final calibration.

```
.\.venv\Scripts\python.exe camera\pick_hsv_color.py
```

```
import cv2, numpy as np
CAMERA_INDEX = 0
def open_camera(index):
    cap = cv2.VideoCapture(index, cv2.CAP_DSHOW)
    if not cap.isOpened(): raise RuntimeError("Camera could not open")
    cap.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc(*'MJPG'))
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1920)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 1080)
    cap.set(cv2.CAP_PROP_FPS, 30)
    cap.set(cv2.CAP_PROP_AUTO_WB, 0)
    cap.set(cv2.CAP_PROP_WB_TEMPERATURE, 5500)
    cap.set(cv2.CAP_PROP_BRIGHTNESS, 100)
    return cap
cap = open_camera(CAMERA_INDEX)
last = None
print("Click the centre of a matte car roof; Q exits")
def click(event,x,y,flags,param):
```

```
    global last
    if event == cv2.EVENT_LBUTTONDOWN and last is not None:
        hsv = cv2.cvtColor(last,cv2.COLOR_BGR2HSV)
        sx = min(last.shape[1]-1, int(x * last.shape[1] / 960))
        sy = min(last.shape[0]-1, int(y * last.shape[0] / 540))
        h,s,v = [int(z) for z in hsv[sy,sx]]
        # Conservative suggested band; red wraps Hue 0/179
        if h <= 10 or h >= 168: ranges = [(0, max(10,h+5)), (max(168,h-5),179)]
        else: ranges = [(max(0,h-8),min(179,h+8))]
        lo_s, hi_s = max(0,s-50), min(255,s+50)
        lo_v, hi_v = max(0,v-60), min(255,v+40)
        print(f"Clicked HSV: [{h}, {s}, {v}]")
        for lo_h,hi_h in ranges:
            print("lower = np.array([%d, %d, %d])"%(lo_h,lo_s,lo_v))
            print("upper = np.array([%d, %d, %d])"%(hi_h,hi_s,hi_v))
        print("Verify sampled ranges against full frame, shadows and track background.")
cv2.namedWindow("HSV picker 960x540")
cv2.setMouseCallback("HSV picker 960x540",click)
while True:
    ok,frame=cap.read()
    if not ok: print("Frame failed"); break
    last=frame.copy(); cv2.imshow("HSV picker 960x540",cv2.resize(frame,(960,540)))
    if cv2.waitKey(1)&0xFF==ord('q'): break
cap.release();cv2.destroyAllWindows()
```

2. Capture reference image & homography (new track only)

Use WARP_W=1800, WARP_H=1000 (18:10 output) and PAD=40 px. Destination rectangle uses the specified padded corners. IMPORTANT: a projective homography needs corresponding points; mapping the four marker CENTRES to this padded rectangle defines a convenient image coordinate system but does not by itself prove the outer walls are exactly scaled or that tags sit on the wall corners. Check the preview, outer barrier and boundary lines; if needed choose destination points from measured track geometry. 100 px/ft is only the intended output scale, not guaranteed by arbitrary tag-centre placement.

File: track/capture_track.py

For full/new track capture only. Press S when all IDs 0–3 are present and the warped preview includes the complete arena; Q exits. Saves track_reference.png and perspective_matrix.npy.

```
.\.venv\Scripts\python.exe track\capture_track.py
```

```
import cv2, numpy as np
from pathlib import Path
P=Path(__file__).resolve().parent; CAMERA_INDEX=0
WARP_W,WARP_H=1800,1000; PAD=40
def open_camera(index):
    cap = cv2.VideoCapture(index, cv2.CAP_DSHOW)
    if not cap.isOpened(): raise RuntimeError("Camera could not open")
    cap.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc(*'MJPG'))
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1920)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 1080)
    cap.set(cv2.CAP_PROP_FPS, 30)
    cap.set(cv2.CAP_PROP_AUTO_WB, 0)
    cap.set(cv2.CAP_PROP_WB_TEMPERATURE, 5500)
    cap.set(cv2.CAP_PROP_BRIGHTNESS, 100)
    return cap
params=cv2.aruco.DetectorParameters(); params.cornerRefinementMethod=cv2.aruco.CORNER_REFINE_SUBPIX
d=cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50); det=cv2.aruco.ArucoDetector(d,params)
cap=open_camera(CAMERA_INDEX); H=None; warped=None
while True:
    ok,frame=cap.read()
    if not ok: break
    view=frame.copy(); corners,ids,_=det.detectMarkers(frame)
    H=None
    if ids is not None:
        cv2.aruco.drawDetectedMarkers(view,corners,ids)
        found={int(i):c[0].mean(axis=0) for c,i in zip(corners,ids.flatten())}
        if all(i in found for i in range(4)):
            src=np.float32([found[i] for i in [0,1,2,3]])
            dst=np.float32([[PAD,PAD],[WARP_W-1-PAD,PAD],[WARP_W-1-PAD,WARP_H-1-PAD],[PAD,WARP_H-1-PAD]])
            H=cv2.getPerspectiveTransform(src,dst); warped=cv2.warpPerspective(frame,H,(WARP_W,WARP_H))
            cv2.imshow("Warp preview 1260x700",cv2.resize(warped,(1260,700)))
    cv2.imshow("Static tags 960x540",cv2.resize(view,(960,540)))
    key=cv2.waitKey(1)&0xFF
    if key==ord('s') and H is not None:
        cv2.imwrite(str(P/"track_reference.png"),warped); np.save(P/"perspective_matrix.npy",H); print("Reference + homography saved")
    if key==ord('q'): break
cap.release();cv2.destroyAllWindows()
```

3. Create drivable-area mask

Keep the existing create_mask.py workflow: paint full drivable surface white, outside black. The script reads dimensions from track_reference.png, so it inherits 1800×1000. Use cyan/yellow centerline display colour if red-car visibility makes the line hard to see; the saved mask itself is unchanged.

File: track/create_mask.py

Use the existing verified script from the previous manual. Left-drag paints valid track; S saves track_mask.png; Q exits. Do not overwrite it with an empty mask.

```
.\.venv\Scripts\python.exe track\create_mask.py
```

```
import cv2, numpy as np
from pathlib import Path
P = Path(__file__).resolve().parent
image = cv2.imread(str(P / "track_reference.png"))
if image is None: raise FileNotFoundError("track_reference.png")
mask = np.zeros(image.shape[:2], np.uint8); brush = 15; mode = None
def mouse(event, x, y, flags, data):
    global mode
    if event == cv2.EVENT_LBUTTONDOWN: mode = 255
    if event == cv2.EVENT_RBUTTONDOWN: mode = 0
    if event in (cv2.EVENT_LBUTTONUP, cv2.EVENT_RBUTTONUP): mode = None
    if mode is not None and (0 <= x < mask.shape[1] and 0 <= y < mask.shape[0]):
        cv2.circle(mask, (x,y), brush, mode, -1)
cv2.namedWindow("Paint mask"); cv2.setMouseCallback("Paint mask", mouse)
while True:
    overlay = image.copy(); overlay[mask == 255] = (0,255,0)
    cv2.imshow("Paint mask", cv2.addWeighted(image,.6,overlay,.4,0))
    key = cv2.waitKey(20) & 0xFF
    if key == ord("s"):
        cv2.imwrite(str(P / "track_mask.png"), mask); print("Mask saved")
    elif key == ord("r"): mask[:] = 0
    elif key in (ord("+"),ord("=")): brush += 3
    elif key == ord("-"): brush = max(2,brush-3)
    elif key == ord("q"): break
cv2.destroyAllWindows()
```

Existing script remains unchanged: use the prior manual's create_mask.py. If it is missing, restore it before proceeding.

4. Extract all valid mask pixels

File: track/extract_pixels.py

Keep the previous verified script unchanged. Saves track_pixels.npy from white mask pixels.

```
.\.venv\Scripts\python.exe track\extract_pixels.py
```

```
import cv2, numpy as np
from pathlib import Path
P = Path(__file__).resolve().parent
mask = cv2.imread(str(P / "track_mask.png"), 0)
if mask is None: raise FileNotFoundError("track_mask.png")
y, x = np.where(mask == 255)
pixels = np.column_stack((x,y))
np.save(P / "track_pixels.npy", pixels)
print("Valid pixels:",len(pixels))
```

5. Draw closed racing centreline

File: track/draw_centerline.py

Keep the previous verified script unchanged. Click centre of drivable width, first point at finish line, follow racing direction; S saves, Q exits. Use cyan/yellow overlay if red conflicts with Car 1.

```
.\.venv\Scripts\python.exe track\draw_centerline.py
```

```
import cv2, numpy as np
from pathlib import Path
```

```
P = Path(__file__).resolve().parent
image = cv2.imread(str(P / "track_reference.png"))
mask = cv2.imread(str(P / "track_mask.png"),0)
if image is None or mask is None: raise FileNotFoundError("Track image/mask")
pts = []
def mouse(event,x,y,flags,data):
    if event == cv2.EVENT_LBUTTONDOWN:
        if mask[y,x] == 255: pts.append((x,y)); print("Added",x,y)
        else: print("Click on white track area")
cv2.namedWindow("Centerline"); cv2.setMouseCallback("Centerline",mouse)
while True:
    view = image.copy()
    if pts:
        for pt in pts: cv2.circle(view,pt,3,(0,0,255),-1)
        if len(pts) > 1: cv2.polylines(view,[np.array(pts,np.int32)],False,(0,0,255),2)
        if len(pts) > 2: cv2.line(view,pts[-1],pts[0],(0,0,255),2)
    cv2.imshow("Centerline",view)
    key = cv2.waitKey(20) & 0xFF
    if key == ord("u") and pts: pts.pop()
    elif key == ord("s") and len(pts) >= 3:
        np.save(P / "centerline_points.npy",np.array(pts+[pts[0]],np.float32))
        print("Centerline saved")
    elif key == ord("q"): break
cv2.destroyAllWindows()
```

6. Calculate dense centreline & progress

File: track/calculate_progress.py

Keep the previous verified script unchanged. Generates dense_centerline.npy, cumulative_distance.npy and lap_length.npy.

```
.\.venv\Scripts\python.exe track\calculate_progress.py
```

```
import numpy as np
from pathlib import Path
P = Path(__file__).resolve().parent
points = np.load(P / "centerline_points.npy")
if len(points) < 4: raise ValueError("Need closed centerline")
dense = [points[0]]; cum = [0.0]; total = 0.0
for a,b in zip(points[:-1],points[1:]):
    length = float(np.linalg.norm(b-a))
    for j in range(1,max(1,int(np.ceil(length)))+1):
        n = max(1,int(np.ceil(length)))
        pt = a + (b-a)*(j/n)
        total += float(np.linalg.norm(pt-dense[-1]))
        dense.append(pt); cum.append(total)
if total <= 0: raise ValueError("Lap length is zero")
np.save(P / "dense_centerline.npy",np.array(dense[:-1]))
np.save(P / "cumulative_distance.npy",np.array(cum[:-1]))
np.save(P / "lap_length.npy",np.array(total))
print("Points:",len(dense)-1,"Lap length:",round(total,2))
```

7. Click-test track progress

File: track/test_progress.py

Keep the previous verified script unchanged. Confirm mask and centreline alignment across corners, boundaries and straights; Q exits.

```
.\.venv\Scripts\python.exe track\test_progress.py
```

```
import cv2, numpy as np
from pathlib import Path
P = Path(__file__).resolve().parent
img = cv2.imread(str(P / "track_reference.png"))
mask = cv2.imread(str(P / "track_mask.png"),0)
line = np.load(P / "dense_centerline.npy")
cum = np.load(P / "cumulative_distance.npy")
length = float(np.load(P / "lap_length.npy"))
def mouse(event,x,y,flags,data):
    if event != cv2.EVENT_LBUTTONDOWN: return
    if mask[y,x] != 255: print((x,y),"OFF TRACK"); return
```

```
    idx = np.argmin(np.sum((line-np.array([x,y]))**2,axis=1))
    print((x,y),"ON TRACK",f"{cum[idx]/length*100:.2f}%")
cv2.namedWindow("Test progress");cv2.setMouseCallback("Test progress",mouse)
while True:
    view = img.copy()
    cv2.polylines(view,[line.astype(np.int32)],True,(0,0,255),2)
    cv2.imshow("Test progress",view)
    if cv2.waitKey(20) & 0xFF == ord("q"): break
cv2.destroyAllWindows()
```

8. Camera-only recalibration

Only use if camera moved but track and corner-tag positions stayed fixed. Do not save a new reference image or repaint mask. This updates only perspective_matrix.npy. Re-check the warped preview and actual alignment. Use the same 1800×1000 dimensions, PAD=40 and subpixel refinement as capture_track.py.

File: track/recalibrate_camera.py

Press S only when all four static IDs are detected and preview aligns with the saved track. Q exits.

```
.\.venv\Scripts\python.exe track\recalibrate_camera.py
```

```
import cv2, numpy as np
from pathlib import Path
P=Path(__file__).resolve().parent; CAMERA_INDEX=0
WARP_W,WARP_H=1800,1000; PAD=40
ref=cv2.imread(str(P/"track_reference.png"))
if ref is None: raise FileNotFoundError("track_reference.png")
def open_camera(index):
    cap = cv2.VideoCapture(index, cv2.CAP_DSHOW)
    if not cap.isOpened(): raise RuntimeError("Camera could not open")
    cap.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc(*'MJPG'))
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1920)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 1080)
    cap.set(cv2.CAP_PROP_FPS, 30)
    cap.set(cv2.CAP_PROP_AUTO_WB, 0)
    cap.set(cv2.CAP_PROP_WB_TEMPERATURE, 5500)
    cap.set(cv2.CAP_PROP_BRIGHTNESS, 100)
    return cap
params=cv2.aruco.DetectorParameters(); params.cornerRefinementMethod=cv2.aruco.CORNER_REFINE_SUBPIX
d=cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50); det=cv2.aruco.ArucoDetector(d,params)
cap=open_camera(CAMERA_INDEX); H=None
while True:
    ok,frame=cap.read()
    if not ok: break
    view=frame.copy(); corners,ids,_=det.detectMarkers(frame); H=None
    if ids is not None:
        cv2.aruco.drawDetectedMarkers(view,corners,ids)
        found={int(i):c[0].mean(axis=0) for c,i in zip(corners,ids.flatten())}
        if all(i in found for i in range(4)):
            src=np.float32([found[i] for i in [0,1,2,3]])
            dst=np.float32([[PAD,PAD],[WARP_W-1-PAD,PAD],[WARP_W-1-PAD,WARP_H-1-PAD],[PAD,WARP_H-1-PAD]])
            H=cv2.getPerspectiveTransform(src,dst); warped=cv2.warpPerspective(frame,H,(WARP_W,WARP_H))
            cv2.imshow("Warp preview 1260x700",cv2.resize(warped,(1260,700)))
    cv2.imshow("Corner tags 960x540",cv2.resize(view,(960,540)))
    key=cv2.waitKey(1)&0xFF
    if key==ord('s') and H is not None: np.save(P/"perspective_matrix.npy",H); print("Homography updated only")
    if key==ord('q'): break
cap.release();cv2.destroyAllWindows()
```

9. Live HSV car tracking + roof parallax correction

No moving ArUco tags are used. Static ArUco IDs 0–3 are only for homography calibration. Segmentation uses the supplied red dual range and blue range; contours are filtered by area. The requested camera settings are attempted but must be verified because some DirectShow drivers ignore auto-WB/brightness properties. The provided roof correction is a simplified radial model, not a full calibrated 3D model; validate at arena edges.

```
.\.venv\Scripts\python.exe track\live_tracking.py
```

```
import cv2, numpy as np
from pathlib import Path
P=Path(__file__).resolve().parent; CAMERA_INDEX=0
CAM_W,CAM_H=1920,1080
PARALLAX_FACTOR=1.0-(0.394/12.0)
MIN_CAR_AREA_PX=800; MAX_CAR_AREA_PX=9000
```

```
CAR_CONFIGS={
 "Car 1 Red":
{"ranges":[(np.array([0,120,50]),np.array([10,255,255])),(np.array([168,120,50]),np.array([179,255,255]))],"color":(0,0,255)},
 "Car 2 Blue":{"ranges":[(np.array([100,120,50]),np.array([128,255,255]))],"color":(255,0,0)},
 # Optional Car 3 Green: add to CAR_CONFIGS after testing
 # "Car 3 Green":{"ranges":[(np.array([42,100,50]),np.array([82,255,255]))],"color":(0,255,0)},
}
def open_camera(index):
    cap=cv2.VideoCapture(index,cv2.CAP_DSHOW)
    if not cap.isOpened(): raise RuntimeError("Camera unavailable")
    cap.set(cv2.CAP_PROP_FOURCC,cv2.VideoWriter_fourcc(*'MJPG'))
    cap.set(cv2.CAP_PROP_FRAME_WIDTH,1920); cap.set(cv2.CAP_PROP_FRAME_HEIGHT,1080); cap.set(cv2.CAP_PROP_FPS,30)
    cap.set(cv2.CAP_PROP_AUTO_WB,0); cap.set(cv2.CAP_PROP_WB_TEMPERATURE,5500); cap.set(cv2.CAP_PROP_BRIGHTNESS,100)
    return cap
def detect_blobs(frame,hsv,kernel):
    found=[]
    for name,cfg in CAR_CONFIGS.items():
        combined=np.zeros(hsv.shape[:2],np.uint8)
        for lower,upper in cfg["ranges"]: combined=cv2.bitwise_or(combined,cv2.inRange(hsv,lower,upper))
        combined=cv2.morphologyEx(combined,cv2.MORPH_OPEN,kernel)
        combined=cv2.morphologyEx(combined,cv2.MORPH_CLOSE,kernel)
        contours,_=cv2.findContours(combined,cv2.RETR_EXTERNAL,cv2.CHAIN_APPROX_SIMPLE)
        valid=[c for c in contours if MIN_CAR_AREA_PX<=cv2.contourArea(c)<=MAX_CAR_AREA_PX]
        if valid:
            c=max(valid,key=cv2.contourArea); m=cv2.moments(c)
            if m["m00"]:
                u=m["m10"]/m["m00"]; v=m["m01"]/m["m00"]
                found.append((name,cfg,(u,v),c))
    return found
ref=cv2.imread(str(P/"track_reference.png")); mask=cv2.imread(str(P/"track_mask.png"),0)
H=np.load(P/"perspective_matrix.npy"); line=np.load(P/"dense_centerline.npy"); cum=np.load(P/"cumulative_distance.npy");
length=float(np.load(P/"lap_length.npy"))
if ref is None or mask is None or length<=0: raise ValueError("Missing track assets")
h,w=mask.shape; kernel=cv2.getStructuringElement(cv2.MORPH_ELLIPSE,(5,5)); cap=open_camera(CAMERA_INDEX)
while True:
    ok,frame=cap.read()
    if not ok: print("Frame failed"); break
    if frame.shape[1]!=CAM_W or frame.shape[0]!=CAM_H: print("WARNING actual camera frame is not 1920x1080")
    hsv=cv2.cvtColor(frame,cv2.COLOR_BGR2HSV); track=ref.copy(); cv2.polylines(track,[line.astype(np.int32)],True,(255,255,0),2)
    for name,cfg,(u,v),contour in detect_blobs(frame,hsv,kernel):
        # Roof centroid -> estimated ground-plane point
        ug=CAM_W/2+(u-CAM_W/2)*PARALLAX_FACTOR; vg=CAM_H/2+(v-CAM_H/2)*PARALLAX_FACTOR
        pt=np.array([[[ug,vg]]],np.float32); x,y=cv2.perspectiveTransform(pt,H)[0,0]; px,py=int(round(x)),int(round(y))
        if not (0<=px<w and 0<=py<h): continue
        cv2.drawContours(frame,[contour],-1,cfg["color"],2)
        status="OFF TRACK"
        if mask[py,px]==255:
            idx=int(np.argmin(np.sum((line-[x,y])**2,axis=1))); progress=cum[idx]/length*100; status=f"{progress:.1f}%"
        cv2.circle(track,(px,py),8,cfg["color"],-1); cv2.putText(track,f"{name}: {status}",(px+10,py-
10),cv2.FONT_HERSHEY_SIMPLEX,.55,cfg["color"],2)
    cv2.imshow("HSV camera 960x540",cv2.resize(frame,(960,540))); cv2.imshow("Mapped track 1260x700",cv2.resize(track,(1260,700)))
    if cv2.waitKey(1)&0xFF==ord('q'): break
cap.release(); cv2.destroyAllWindows()
```

10. HSV lap counter + jump recovery

Prototype lap counter for HSV blobs. It uses the same contour extraction, perspective matrix, mask and roof parallax correction. MAX_JUMP=18%, finish windows 88%/12%. On a rejected jump, previous progress is updated to prevent a permanent lock; this improves recovery but may also hide a real shortcut/reverse event. This logic is not competition-grade until tested and replaced with timestamp-aware motion and physical checkpoint gates.

```
.\.venv\Scripts\python.exe track\lap_counter.py
```

```
import cv2, numpy as np
from pathlib import Path
P=Path(__file__).resolve().parent; CAMERA_INDEX=0
CAM_W,CAM_H=1920,1080
PARALLAX_FACTOR=1.0-(0.394/12.0)
MIN_CAR_AREA_PX=800; MAX_CAR_AREA_PX=9000
CAR_CONFIGS={
 "Car 1 Red":
{"ranges":[(np.array([0,120,50]),np.array([10,255,255])),(np.array([168,120,50]),np.array([179,255,255]))],"color":(0,0,255)},
 "Car 2 Blue":{"ranges":[(np.array([100,120,50]),np.array([128,255,255]))],"color":(255,0,0)},
 # Optional Car 3 Green: add to CAR_CONFIGS after testing
 # "Car 3 Green":{"ranges":[(np.array([42,100,50]),np.array([82,255,255]))],"color":(0,255,0)},
}
def open_camera(index):
    cap=cv2.VideoCapture(index,cv2.CAP_DSHOW)
```

```
    if not cap.isOpened(): raise RuntimeError("Camera unavailable")
    cap.set(cv2.CAP_PROP_FOURCC,cv2.VideoWriter_fourcc(*'MJPG'))
    cap.set(cv2.CAP_PROP_FRAME_WIDTH,1920); cap.set(cv2.CAP_PROP_FRAME_HEIGHT,1080); cap.set(cv2.CAP_PROP_FPS,30)
    cap.set(cv2.CAP_PROP_AUTO_WB,0); cap.set(cv2.CAP_PROP_WB_TEMPERATURE,5500); cap.set(cv2.CAP_PROP_BRIGHTNESS,100)
    return cap
def detect_blobs(frame,hsv,kernel):
    found=[]
    for name,cfg in CAR_CONFIGS.items():
        combined=np.zeros(hsv.shape[:2],np.uint8)
        for lower,upper in cfg["ranges"]: combined=cv2.bitwise_or(combined,cv2.inRange(hsv,lower,upper))
        combined=cv2.morphologyEx(combined,cv2.MORPH_OPEN,kernel)
        combined=cv2.morphologyEx(combined,cv2.MORPH_CLOSE,kernel)
        contours,_=cv2.findContours(combined,cv2.RETR_EXTERNAL,cv2.CHAIN_APPROX_SIMPLE)
        valid=[c for c in contours if MIN_CAR_AREA_PX<=cv2.contourArea(c)<=MAX_CAR_AREA_PX]
        if valid:
            c=max(valid,key=cv2.contourArea); m=cv2.moments(c)
            if m["m00"]:
                u=m["m10"]/m["m00"]; v=m["m01"]/m["m00"]
                found.append((name,cfg,(u,v),c))
    return found
ref=cv2.imread(str(P/"track_reference.png")); mask=cv2.imread(str(P/"track_mask.png"),0); H=np.load(P/"perspective_matrix.npy")
line=np.load(P/"dense_centerline.npy"); cum=np.load(P/"cumulative_distance.npy"); length=float(np.load(P/"lap_length.npy"))
if ref is None or mask is None or length<=0: raise ValueError("Missing track assets")
h,w=mask.shape; kernel=cv2.getStructuringElement(cv2.MORPH_ELLIPSE,(5,5)); cap=open_camera(CAMERA_INDEX)
CHECKPOINTS=[25.,50.,75.]; MAX_JUMP=18.; FINISH_HIGH=88.; FINISH_LOW=12.
cars={name:{"laps":0,"progress":None,"previous":None,"cp":0,"armed":False,"status":"WAITING"} for name in CAR_CONFIGS}
def update(name,p):
    c=cars[name]; prev=c["previous"]
    if prev is None:
        c["previous"]=c["progress"]=p; c["armed"]=p<=FINISH_LOW; c["status"]="RACING" if c["armed"] else "MOVE TO START"; return
    delta=(p-prev)%100.
    if delta>MAX_JUMP:
        c["previous"]=p  # recover next frame; avoid permanent jump lockup
        c["status"]="JUMP/REVERSE REJECTED"; return
    c["previous"]=c["progress"]=p
    if not c["armed"]:
        if p<=FINISH_LOW: c["armed"]=True; c["cp"]=0; c["status"]="RACING"
        else: c["status"]="MOVE TO START"
        return
    if p>=prev and c["cp"]<3 and prev<CHECKPOINTS[c["cp"]]<=p: c["cp"]+=1
    if prev>=FINISH_HIGH and p<=FINISH_LOW and p<prev:
        if c["cp"]==3: c["laps"]+=1; print(name,"LAP",c["laps"])
        else: print(name,"finish rejected: checkpoints incomplete")
        c["cp"]=0
    c["status"]="RACING"
while True:
    ok,frame=cap.read()
    if not ok: print("Frame failed"); break
    hsv=cv2.cvtColor(frame,cv2.COLOR_BGR2HSV); track=ref.copy(); cv2.polylines(track,[line.astype(np.int32)],True,(255,255,0),2)
    seen=set()
    for name,cfg,(u,v),contour in detect_blobs(frame,hsv,kernel):
        ug=CAM_W/2+(u-CAM_W/2)*PARALLAX_FACTOR; vg=CAM_H/2+(v-CAM_H/2)*PARALLAX_FACTOR
        x,y=cv2.perspectiveTransform(np.array([[[ug,vg]]],np.float32),H)[0,0]; px,py=int(round(x)),int(round(y))
        if not (0<=px<w and 0<=py<h): cars[name]["status"]="OUTSIDE IMAGE"; continue
        seen.add(name)
        if mask[py,px]!=255: cars[name]["status"]="OFF TRACK"; continue
        idx=int(np.argmin(np.sum((line-[x,y])**2,axis=1))); progress=float(cum[idx]/length*100); update(name,progress)
        cv2.circle(track,(px,py),8,cfg["color"],-1)
    for j,(name,c) in enumerate(cars.items()):
        status=c["status"] if name in seen else "NOT DETECTED"
        prog="--" if c["progress"] is None else f"{c['progress']:.1f}%"
        cv2.putText(track,f"{name} Laps:{c['laps']} Progress:{prog} CP:{c['cp']}/3
{status}",(12,28+j*28),cv2.FONT_HERSHEY_SIMPLEX,.42,(255,255,255),1)
    cv2.imshow("HSV camera 960x540",cv2.resize(frame,(960,540))); cv2.imshow("Lap counter 1260x700",cv2.resize(track,(1260,700)))
    if cv2.waitKey(1)&0xFF==ord('q'): break
cap.release(); cv2.destroyAllWindows()
```

11. EXACT PowerShell execution sequences

All commands run from the gazebo root. Replace CAMERA_INDEX in scripts after checking camera_detect.py. Close each program before running the next.

A. Full Rebuild — new/larger track or corner tags moved

```
# 1. Check camera index and actual 1080p feed
.\.venv\Scripts\python.exe camera\camera_detect.py
.\.venv\Scripts\python.exe camera\camera_test.py
```

```
# 2. Capture new track image + homography; press S then Q
.\.venv\Scripts\python.exe track\capture_track.py
# 3. Repaint new drivable surface; press S then Q
.\.venv\Scripts\python.exe track\create_mask.py
.\.venv\Scripts\python.exe track\extract_pixels.py
# 4. Draw closed centreline; first point is finish; press S then Q
.\.venv\Scripts\python.exe track\draw_centerline.py
.\.venv\Scripts\python.exe track\calculate_progress.py
.\.venv\Scripts\python.exe track\test_progress.py
# 5. Calibrate car colours in current outdoor light; click each roof, Q
.\.venv\Scripts\python.exe camera\pick_hsv_color.py
# 6. Update CAR_CONFIGS if required, then verify tracking
.\.venv\Scripts\python.exe track\live_tracking.py
# 7. Slow validation lap test
.\.venv\Scripts\python.exe track\lap_counter.py
```

B. Camera position changed; track and corner tags unchanged

```
# 1. Verify camera index/mode if needed
.\.venv\Scripts\python.exe camera\camera_detect.py
.\.venv\Scripts\python.exe camera\camera_test.py
# 2. Update only homography; preview must align; press S then Q
.\.venv\Scripts\python.exe track\recalibrate_camera.py
# 3. If sunlight has changed, sample roofs and update CAR_CONFIGS
.\.venv\Scripts\python.exe camera\pick_hsv_color.py
# 4. Verify mapped cars, then lap counter
.\.venv\Scripts\python.exe track\live_tracking.py
.\.venv\Scripts\python.exe track\lap_counter.py
```

C. Same setup / quick HSV sunlight check

```
# Quick mode/feed verification; Q exits
.\.venv\Scripts\python.exe camera\camera_test.py
# If lighting/time-of-day changed, click each car roof; Q exits
.\.venv\Scripts\python.exe camera\pick_hsv_color.py
# Update CAR_CONFIGS in both scripts if necessary, then run:
.\.venv\Scripts\python.exe track\live_tracking.py
.\.venv\Scripts\python.exe track\lap_counter.py
```

If track is new or bigger, use A, not B. A camera-only recalibration cannot adapt a saved mask/centreline to a different arena. If camera moved and sunlight changed, do both recalibration and HSV sampling.

12. Troubleshooting

Problem	Check / action
Camera not 1920×1080	Print actual frame shape; confirm MJPEG support, USB port/cable, DirectShow driver. Requested CAP_PROP values may be ignored.
Camera index changed	Run camera_detect.py; identify FHDWC300 feed; update CAMERA_INDEX in scripts.
ArUco corner IDs missing	Use DICT_4X4_50, IDs 0–3; verify 18 cm marker, matte substrate, focus and sunlight exposure.
Sunlight glare / white-out	Avoid glossy surfaces; shade lens if possible; reduce exposure via supported driver controls. Brightness=100 is driver-specific and may increase clipping rather than prevent it; inspect histogram and tune experimentally.
White card blooms / markers wash out	Use specified matte light-grey substrate, avoid specular glare, adjust exposure if supported; verify IDs in direct sun.
HSV misses car in shadow	Run pick_hsv_color.py under both sun and shadow; widen tested S/V/H ranges; consider per-car multiple ranges and ROI.
Background matches car colour	Repaint floor matte charcoal; ensure white lines do not enter blob; tighten HSV and use geometry/temporal association.
Two cars merge or switch labels	Current prototype chooses largest contour per colour and does not do robust identity association. Separate hues, enforce spacing, add tracking/assignment before event.
Blob area rejected	Inspect contour area at full resolution; tune MIN_CAR_AREA_PX/MAX_CAR_AREA_PX. Expected projected car size is approximate, not fixed.
Parallax off-track at edge	0.967 radial correction is approximate. Verify roof-centroid/wheel-ground offsets across the arena; calibrate with measured ground

	points. Do not keep adjusting until mask aligns without ground-truth checks.
Progress jump / reverse warning	MAX_JUMP=18 is a heuristic; jump path updates previous for recovery. It can also mask a shortcut/reverse event; use timestamp-aware motion and physical checkpoint gates for official use.
Mask/centreline no longer aligns	If only camera moved, recalibrate. If track/tag layout changed, run full rebuild and repaint mask/redrawn centreline.
Windows are too large	Camera preview is resized to 960×540 and mapped track to 1260×700; output image itself stays 1920×1080 and warp 1800×1000.

13. Final Outdoor Event-Day Readiness Checklist

☐ Secure the FHDWC300 at measured 12.0 ft, centred and vertically downward; ensure all track and four corner tags fit in the actual 1920×1080 image.

☐ Verify actual 1920×1080 frames and stable FPS; verify MJPEG and white balance/exposure properties are accepted by the driver. Record tested settings.

☐ Rebuild the reference, mask and centreline for the final 18×10 ft (or final measured) arena. Never reuse the tabletop track mask/centreline.

☐ Confirm 18 cm tags, IDs 0–3, correct order, matte cardstock, no clipping, and reliable detection in direct sun.

☐ Test roof colours in sun, cloud and shadow with pick_hsv_color.py. Update CAR_CONFIGS for all cars and validate full-frame false positives.

☐ Validate parallax correction using measured ground truth at centre, edges and corners. The supplied 0.967 factor is a first-order approximation only.

☐ Test each car at walking pace and 2–4 m/s, then higher speeds only after frame continuity and detection accuracy are measured.

☐ Test occlusion, overlapping cars, shadows, missed frames, reverse motion, off-track exits, finish-line crossings, resets and laptop/network interruptions.

☐ Replace prototype lap counter with time-aware tracking, robust car identity association, directed physical checkpoint gates and explicit race state before official scoring.

☐ Log timestamps, positions, HSV masks, detected contours, checkpoint transitions, laps and errors. Compare automated laps with independent video/manual ground truth.

☐ Keep backups of all calibrated files, spare webcam/cable, printed tags, laptop/power, and a manual scoring fallback.

Engineering assumptions and cautions

This manual follows the requested geometry and HSV strategy, but the supplied optical footprint, pixel density and car pixel area are estimates and should be measured on the real camera/arena. A projective transform from four marker centres cannot independently guarantee metric dimensions unless their physical positions and target coordinates are known. The requested radial parallax factor is approximate; roof height, lens distortion, camera tilt, and point position all affect error. DirectShow exposure/white-balance properties are hardware/driver-dependent. The current largest-contour HSV implementation is a prototype, not a validated multi-car event scorer; use ground-truth testing before official competition.